



Методическое пособие
по выполнению домашнего
задания курса
**Инфраструктурная платформа на
основе Kubernetes**

Создание собственного CRD

Содержание

1.	Введение	3
2.	Цели домашнего задания	4
3.	Описание домашнего задания	5
4.	Пошаговая инструкция выполнения домашнего задания	7
5.	Сдача задания	9
6.	Критерий оценки	10
7.	Рекомендуемые источники	11

1. Введение

CustomResourceDefinition

CRD –это инструмент, который позволяет расширять API Kubernetes, добавляя собственные, пользовательские ресурсы.

Эти определения не просто новые названия для существующих концепций; они создают новые типы ресурсов, которые Kubernetes будет распознавать и обрабатывать так же, как и встроенные ресурсы, такие как Pods, Deployments и Services.

CRD функционируют путем определения новых типов ресурсов. Сначала создается описание ресурса, которое говорит Kubernetes, какие поля будут доступны и как они должны быть обработаны. Как только CRD определен и добавлен в кластер Kubernetes, он становится доступен через API Kubernetes, позволяя создавать, обновлять и удалять объекты с использованием этого нового определения.

В задании мы создадим CRD для установки mysql в кластер, опишем какие параметры может принимать этот ресурс и правила валидации этих параметров.

Operator

это под в Kubernetes-кластере, приложение которое по большому счету, следит за установкой и изменениями CRD. Его сила в том, что он может взаимодействовать как с API кластера, так и с внешними ресурсами. А самое главное, он может приводить систему в нужное состояние своими собственными силами. От разработчиков требуется только декларативное описание нужного стейта.

В задании вы сможете использовать уже готовый оператор, либо написать свой собственный (задание с *)

2. Цели домашнего задания



- 1) Научиться создавать и конфигурировать CR и CRD ресурсы.
- 2) Научиться создавать и устанавливать в кластер собственные Operators



3. Описание домашнего задания



Подготовка к выполнению домашнего задания

- Создайте branch `kubernetes-operators` - данное домашнее задание будет выполняться в этой ветке.
- Создайте папку `kubernetes-operators` - все файлы, которые у вас получается во время выполнения данного ДЗ необходимо поместить в эту папку.

Рекомендуемые источники



- Официальная документация по [CRD](#)
- Документация по паттерну [Operator](#)
- Пример создания и конфигурирования [CRD](#)
- Для написания собственного оператора можно рассмотреть фреймворк
 - [Kopf](#) - если вы знакомы с python. Также можете посмотреть как пример образ контейнера, используемого в задании без * - для написания оператора в нем как раз использовался Kopf
 - [OperatorFramework](#) – предоставляет возможность реализовать оператор на [Go](#) либо на [Ansible](#)
 - Если вы уже работали с каким то другим фреймворком для создания операторов – можете реализовать на нем



4. Пошаговая инструкция выполнения домашнего задания



- Создать манифест объекта CustomResourceDefinition со следующими параметрами:
 - Объект уровня namespace
 - Api group – otus.homework
 - Kind – MySQL
 - Plural name – mysqls
 - Версия – v1
- Объект должен иметь следующие **обязательные** атрибуты и правила их валидации(все поля строковые):
 - Image – определяет docker-образ для создания
 - Database – имя базы данных
 - Password – пароль от БД
 - Storage_size – размер хранилища под базу
- Создать манифесты ServiceAccount, ClusterRole и ClusterRoleBinding, описывающий сервис аккаунт с полными правами на доступ к api серверу
- Создать манифест deployment для оператора, указав созданный ранее ServiceAccount и образ [roflmaoinmysoul/mysql-operator:1.0.0](#)
- Создать манифест кастомного объекта kind: MySQL валидный для применения (см. атрибуты CRD созданного ранее)
- Применить все манифесты и убедиться, что CRD создан, оператор работает и при создании кастомного ресурса типа MySQL создает Deployment с указанным образом mysql, service для него, PV и PVC. При удалении объекта типа MySQL удаляются все созданные для него ресурсы



4. Пошаговая инструкция выполнения домашнего задания (продолжение)



Задание с *

- Изменить манифест ClusterRole, описав в нем минимальный набор прав доступа необходимые для нашего CRD и убедиться что функциональность не пострадала
 - Управление сами ресурсом CRD
 - Создание и удаление ресурсов типа Service, PV, PVC

Задание с **

- Создать свой оператор, который будет реализовывать следующий функционал
 - При создании в кластере объектов с типом MySQL (mysqls.otus.homework/v1) будет создавать deployment с заданным образом mysql, сервис типа ClusterIP, PV и PVC заданного размера
 - При удалении объекта с типом MySQL будет удалять ранее созданные ресурсы



5. Сдача задания



- Добавьте все получившиеся файлы в ветку **kubernetes-operators**
- Создайте Pull Request к ветке master
- Заполните описание PR по шаблону
- **Не мерджите** PR самостоятельно
- Если у вас возникли вопросы при выполнении ДЗ и необходима консультация преподавателей – добавьте к PR метку **Review Required**
- В личном кабинете Otus сдайте ДЗ на проверку, указав ссылку на Pull Request



6. Критерий оценивания



- 0 баллов – задание не выполнено или выполнено не полностью
- 1 балл – все основные задания выполнены полностью
- 2 балла – все задания включая дополнительные выполнены полностью



7. Рекомендуемые источники



- Официальная документация по [CRD](#)
- Документация по паттерну [Operator](#)
- Пример создания и конфигурирования [CRD](#)
- Для написания собственного оператора можно рассмотреть фреймворк
 - [Korf](#) - если вы знакомы с python. Также можете посмотреть как пример образ контейнера, используемого в задании без * - для написания оператора в нем как раз использовался korf
 - [OperatorFramework](#) – предоставляет возможность реализовать оператор на [Go](#) либо на [Ansible](#)
 - Если вы уже работали с каким то другим фреймворком для создания операторов – можете реализовать на нем

